MVP API Contract v1 (Thin Slice)

Global Decisions

- **Versioning:** All endpoints prefixed `/api/v1/` to protect against breaking changes (multi-tenancy, AIBOM later).
- **ID prefixing:** Scan IDs use `scn_` prefix (e.g., `scn_9b3a2f8c`) for easier tracing across Postgres and Redis.
- **Mock auth:** Frontend sends `Authorization: Bearer dev_placeholder` on every request. Backend ignores it for now. Zero UI changes when real Keycloak is wired in.
- **Errors:** All error responses use one shape:

```
json

{ "error": { "code": "SCAN_NOT_FOUND", "message": "No scan with id scn_9b3a2f8c
```

---

1. Initiate Scan

`POST /api/v1/scans`

Accepts a repo link, queues the job via BullMQ, returns a tracking ID immediately. Returns **202 Accepted** (not 200) to signal the work is queued, not done. File uploads are out of scope for this slice.

Request

```
json

{ "repoUrl": "https://github.com/expressjs/express" }
```

Response 202

```
json

{
  "id": "scn_9b3a2f8c",
  "status": "queued",
  "repoUrl": "https://github.com/expressjs/express",
  "createdAt": "2026-07-22T06:15:00Z"
}
```

Errors: 400 `INVALID_REPO_URL` if the URL is not a GitHub/GitLab repo URL.

---

2. Get Scan (status + polling)

`GET /api/v1/scans/{scanId}`

Returns the scan resource itself. Serves both the poll loop and page-load recovery after refresh (the scan id lives in the frontend URL, so this endpoint must return enough to rebuild the page).

Frontend polls every 3s, backing off to 5s then 10s. Keep this payload light; no components here.

Response 200 (in progress)

json


```
{
  "id": "scn_9b3a2f8c",
  "status": "scanning",
  "phase": 3,
  "phaseCount": 5,
  "repoUrl": "https://github.com/expressjs/express",
  "createdAt": "2026-07-22T06:15:00Z",
  "updatedAt": "2026-07-22T06:16:42Z",
  "error": null
}
```


Response 200 (failed)

json


```
{
  "id": "scn_9b3a2f8c",
  "status": "failed",
  "phase": 2,
  "phaseCount": 5,
  "repoUrl": "https://github.com/expressjs/express",
  "createdAt": "2026-07-22T06:15:00Z",
  "updatedAt": "2026-07-22T06:15:31Z",
  "error": "Repository could not be cloned. It may be private or the URL may be
}
```


Status enum (ordered): `queued` → `cloning` → `scanning` → `enriching` → `completed`, plus terminal `failed` from any state. `phase`/`phaseCount` give the progress bar something ordinal to render; Sarthak maps status strings to labels, nothing more.

Backend obligations: hard job timeout (30 min) marks the scan `failed` with a timeout message; BullMQ stalled-job detection on, so a dead worker never leaves a scan stuck in `scanning`.

Errors: 404 `SCAN_NOT_FOUND`.

---

3. Get Components (paginated from day one)

`GET /api/v1/scans/{scanId}/components?page=1&limit=50`

Called after status is `completed`. Re-callable any time (refresh, revisit); backend treats it as a plain idempotent read. Paginated now, not later: 8,000-component SBOMs are a known risk, and adding pagination mid-sprint changes the contract both devs build against.

Payload is our internal format-agnostic model, not raw CycloneDX/SPDX.

Response 200

json

```
{
  "scanId": "scn_9b3a2f8c",
  "page": 1,
  "limit": 50,
  "total": 62,
  "components": [
    {
      "id": "cmp_4f2e91aa",
      "name": "accepts",
      "version": "1.3.8",
      "type": "library",
      "purl": "pkg:npm/accepts@1.3.8",
      "licenses": ["MIT"],
      "direct": true
    }
  ]
}
```

Field notes: `type` is the extensible component-type field (AIBOM slots in later). `licenses` is an array because dual licensing exists. `direct` distinguishes direct vs transitive deps; cheap to include now, useful in the UI immediately.

Errors: 404 `SCAN_NOT_FOUND`; 409 `SCAN_NOT_COMPLETE` if called before `completed`.

---

4. Export SBOM

`GET /api/v1/scans/{scanId}/export?format=cyclonedx`

The demo's final step: download a valid CycloneDX file. Contract exists now even if the implementation lands in week 2.

Query: `format` = `cyclonedx` (this slice) | `spdx` (later, contract reserved).

Response 200: the raw SBOM document.

- `Content-Type: application/json`
- `Content-Disposition: attachment; filename="scn_9b3a2f8c.cyclonedx.json"`

Errors: 404 `SCAN_NOT_FOUND`; 409 `SCAN_NOT_COMPLETE`; 400 `UNSUPPORTED_FORMAT`.

Change Log vs Draft

1. Components endpoint paginated from day one (was "flat array, pagination in mind").
2. Export endpoint added (was missing; it is the demo's success criterion).
3. Full JSON request/response shapes added for every endpoint.
4. "Hit exactly once" removed; results are idempotent reads, re-callable on refresh.
5. `/status` sub-resource replaced by `GET /scans/{scanId}` returning the scan resource, so one endpoint serves polling and refresh recovery. Renamed `/results` to `/components` to match what it returns.
6. Standard error shape and per-endpoint error codes added.
7. `phase`/`phaseCount` added so the progress bar renders from data, not inferred state names.